

BINARY CLASSIFICATION USING KNN CLASSIFIER

Ankit Saini (2522MA01)
Tiyas Chakrabortty (2522MA02)
Tanu Goliyan (2621MA05)
Murari Ray (2621MA07)

Department of Mathematics
Indian Institute of Technology Patna

16-08-2026

1 Introduction

This task implements a K-Nearest Neighbors (KNN) classifier for multi-class image classification on the CIFAR-10 dataset. The notebook explicitly implements the KNN classifier and the required distance functions from scratch rather than using a pre-built KNN estimator. The experiment evaluates five distance measures for K values 3, 4, 9, 20, and 47.

CIFAR-10 contains 10 image classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck. Each image is 32×32 pixels with three RGB channels, giving

$$32 \times 32 \times 3 = 3072$$

stored features per image.

2 Objectives

The main objectives are to implement KNN from scratch, compare different distance metrics, study the effect of K on classification accuracy, select the best configuration, and evaluate the selected model using a confusion matrix, precision, and recall.

3 Dataset and Preprocessing

The notebook loads the CIFAR-10 Python batches and reports 50,000 training images and 10,000 testing images in the complete dataset. For the brute-force KNN experiment, it selects 5,000 training images and 1,000 test images to keep computation practical in Colab. The random selection uses seed 42.

Pixel values originally range from 0 to 255. They are normalized by dividing by 255, so the values used for distance calculations lie between 0 and 1.

4 Methodology

4.1 KNN Classifier Procedure

For every test image, the implementation calculates its distance from all selected training images, sorts the distances, selects the K nearest samples, examines their labels, and assigns the most frequent class among those neighbors.

4.2 Distance Measures

Metric	Description used in the experiment
Euclidean	Straight-line distance based on squared feature differences.
Manhattan	Sum of absolute feature differences.
Minkowski	Generalized distance using a power parameter.
Cosine	Measures angular similarity between feature vectors; implemented as a distance.
Hamming	Measures the proportion of differing feature values.

Table 1: Distance measures used in the experiment.

4.3 Evaluation

Accuracy is computed as the mean of correct predictions. For the best configuration, the notebook also constructs a 10×10 confusion matrix and calculates class-wise precision and recall from true positives, false positives, and false negatives.

5 Experimental Setup

Table 2: Experimental parameters and dataset configuration.

Parameter	Value
Dataset	CIFAR-10
Complete training set	50,000 images
Complete test set	10,000 images
Experimental training subset	5,000 images
Experimental test subset	1,000 images
Image representation	3072 features ($32 \times 32 \times 3$)
Normalization	Pixel value / 255
K values	3, 4, 9, 20, 47
Distance metrics	Euclidean, Manhattan, Minkowski, Cosine, Hamming

6 Experimental Results

6.1 Accuracy for Different K and Distance Metrics

Table 3: Testing accuracy obtained for different values of K and distance metrics.

K	Euclidean	Manhattan	Minkowski	Cosine	Hamming
3	0.192	0.252	0.116	0.096	0.208
4	0.194	0.252	0.122	0.094	0.228
9	0.224	0.278	0.134	0.12	0.264
20	0.230	0.268	0.136	0.116	0.254
47	0.234	0.26	0.128	0.102	0.246

6.2 Best Configuration

The highest testing accuracy in the recorded experiment is 0.303 (30.3%), obtained using cosine distance with $K = 9$. The best result for each metric is: Euclidean – $K = 9$, accuracy 0.224; Manhattan – $K = 9$, accuracy 0.278; Minkowski – $K = 20$, accuracy 0.136; Cosine – $K = 9$, accuracy 0.12; Hamming – $K = 9$, accuracy 0.264.

Table 4: Best K and accuracy for each distance metric.

Metric	Best K	Best Accuracy
Euclidean	9	0.224
Manhattan	9	0.278
Minkowski	20	0.136
Cosine	9	0.120
Hamming	9	0.264

7 Evaluation of the Best Model

After selecting **Manhattan** distance with $K = 9$, the notebook generates predictions for all 1,000 selected test images. The resulting confusion matrix is shown below. Rows represent actual classes and columns represent predicted classes.

$$\begin{bmatrix} 35 & 0 & 7 & 1 & 4 & 0 & 2 & 1 & 7 & 0 \\ 13 & 3 & 8 & 5 & 7 & 0 & 1 & 0 & 2 & 2 \\ 14 & 0 & 22 & 4 & 5 & 0 & 4 & 1 & 1 & 0 \\ 2 & 0 & 22 & 5 & 8 & 1 & 8 & 0 & 2 & 1 \\ 11 & 2 & 8 & 2 & 12 & 0 & 3 & 2 & 0 & 0 \\ 7 & 1 & 12 & 2 & 8 & 6 & 7 & 3 & 2 & 0 \\ 11 & 0 & 15 & 3 & 3 & 1 & 20 & 1 & 0 & 0 \\ 10 & 1 & 5 & 5 & 10 & 1 & 5 & 6 & 3 & 1 \\ 17 & 1 & 4 & 2 & 4 & 1 & 1 & 0 & 26 & 1 \\ 22 & 5 & 9 & 5 & 1 & 2 & 1 & 0 & 7 & 41 \end{bmatrix}$$

7.1 Class-wise Precision and Recall

Table 5: Class-wise Precision and Recall.

Class	Precision	Recall
airplane	0.250	0.610
automobile	0.230	0.070
bird	0.200	0.430
cat	0.150	0.100
deer	0.190	0.300
dog	0.500	0.120
frog	0.380	0.370
horse	0.430	0.130
ship	0.520	0.460
truck	0.440	0.070

8 Discussion and Observations

The experiment demonstrates that changing K does not necessarily improve accuracy. For example, cosine distance achieves its highest recorded accuracy at $K = 9$, while its accuracy decreases at $K = 47$. Similarly, each distance metric has a different K at which it performs best.

The best overall result, 27.8% accuracy, is relatively modest. The notebook explains that CIFAR-10 is more difficult for brute-force KNN than a low-dimensional binary classification problem because the images are represented by 3072-dimensional pixel features and there are ten classes. The confusion matrix also shows substantial class confusion.

The results should be interpreted as the outcome of the selected 5,000-training/1,000-test subset experiment. They are not a claim about a full 50,000/10,000 brute-force KNN run.

9 Limitations and Computational Considerations

The complete CIFAR-10 dataset contains 50,000 training and 10,000 test images, but a pure brute-force KNN implementation requires a very large number of distance calculations. The provided notebook therefore uses 5,000 training images and 1,000 test images for a practical Colab experiment. Increasing the subset to the full dataset may be extremely slow and can exceed available memory or execution time.

The classifier works directly with raw pixel features. The notebook does not apply learned image features or dimensionality reduction before KNN, so the representation remains a high-dimensional vector of pixel values.

10 Conclusion

A multi-class KNN classifier was implemented from scratch and evaluated on a practical subset of CIFAR-10. Five distance metrics and five K values were compared. The

best recorded configuration was **Manhattan** distance with $K = 9$, producing a testing accuracy of (27.8%). The experiment confirms that both the choice of distance metric and the value of K affect KNN performance.

The study also highlights the computational cost of brute-force KNN on high-dimensional image data. The results provide a baseline for understanding KNN behavior on CIFAR-10 and for comparing future improvements in pre-processing, feature representation, or more efficient nearest-neighbor search.

11 Key Experimental Values

Table 6: Key Experimental Values.

Item	Recorded value
Training subset	5,000 images
Test subset	1,000 images
Number of features/image	3,072
Number of classes	10
Best K	9
Best distance metric	Manhattan
Best testing accuracy	0.278(27.8%)